BufSim: SQL Query Engine with Buffer Pool Simulation

Team Members:

23CS10010 Bethu Kriti

23CS10028 Jindam Sreenath

23CS10048 Nutubilli Gayatri

23CS30019 E. Sai Vishesh Sharma

23CS30034 M Subhasish Varma

April 15, 2026

Contents

1 Introduction **4**

2 Objective **4**

3 System Architecture **5**

4 Methodology **6**

4.1 Page Structure 6

4.2 Buffer Pool 7

4.3 Replacement Policies 7

4.3.1 Least Recently Used (LRU) 7

4.3.2 Most Recently Used (MRU) 7

4.3.3 Clock (Second Chance) 8

5 Working of the System **8**

5.1 Query Processing Pipeline 8

5.2 I/O Cost Analysis 9

6 Implementation **9**

6.1 File Organisation 9

6.2 Buffer Manager 9

6.3 SQL Parser 10

6.4 Query Executor 10

7 User Interface **11**

7.1 Terminal REPL 11

7.2 HTML-Based Interactive UI 11

8 Experimental Setup **11**

9 Results **14**

9.1 Sample Output 14

9.2 Buffer Statistics Format 15

9.3 Policy Comparison 15

10 Discussion **15**

11 Challenges Faced **16**

11.1 Designing a Correct Buffer Manager 16

11.2 Managing Binary Storage 16

11.3 SQL Parsing 16

11.4 Cache Invalidation on Writes 16
11.5 Join Strategy Selection 16
11.6 HTML UI Synchronisation 16

12 Teamwork and Collaboration **16**

13 Future Improvements **17**

14 Conclusion **17**

15 References **18**

1 Introduction

Database systems process large volumes of data stored on disk. Since disk access is significantly slower than main memory access, database systems use a component called the **Buffer Manager**. The buffer manager maintains a limited number of memory frames called the **buffer pool** to temporarily store pages read from disk.

When the buffer pool becomes full, the system must decide which page should be replaced to load a new page. This decision is made using a **buffer replacement policy**.

In this project, **BufSim** is a from-scratch database engine written in C++17 that simulates a page-level buffer pool with three replacement policies (LRU, MRU, CLOCK) and a full SQL front-end. The engine processes queries through a REPL shell, logging all results to `output.txt`.

The system implements a small but complete SQL query engine supporting:

- `CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`
- `INSERT`, `SELECT`, `UPDATE`, `DELETE`
- `JOIN` (Nested-Loop Join and Block Nested-Loop Join)
- Aggregation: `COUNT(*), SUM, AVG, MIN, MAX`
- Clauses: `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, `OFFSET`
- `SHOW TABLES`, `DISTINCT`, and a rich operator set

The project allows analysis of how different buffer management strategies affect the performance of query execution.

2 Objective

The main objectives of this project are:

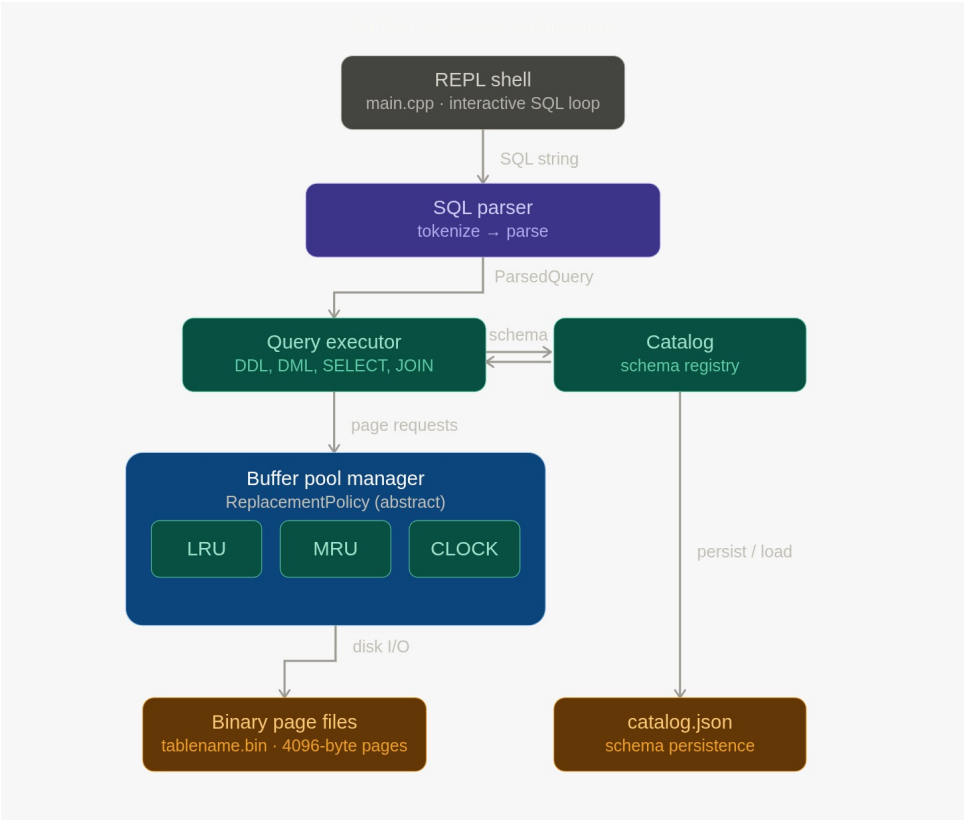
- To simulate a small buffer pool for database query processing.
- To implement and compare three buffer replacement policies:
  - Least Recently Used (LRU)
  - Most Recently Used (MRU)
  - Clock (Second Chance)
- To measure the performance of each strategy based on:
  - Page accesses

  - Disk reads
  - Page hits and page misses
  - Cache hit rate

- To build a SQL query processor supporting DDL, DML, aggregation, join operations, and advanced clauses.
- To provide I/O cost analysis (NLJ, BNLJ, sequential scan) and per-query hints.
- To offer an interactive HTML-based UI (`index.html`) for in-browser query execution alongside the terminal REPL.
- To demonstrate how buffer management impacts the efficiency of query execution.

3 System Architecture

The system is designed as a small relational query engine with an integrated buffer manager.



The architecture consists of the following major components:

1. **SQL Parser** (`sqlParser.hpp/cpp`)
   Reads the SQL query and converts it into a structured *ParsedQuery*. Supports

DDL (`CREATE`, `DROP`, `ALTER`), DML (`INSERT`, `UPDATE`, `DELETE`), `SELECT` with full clause support, and `JOIN`.

2. **Query Executor** (`queryExecutor.hpp/cpp`)
   Interprets the *ParsedQuery* and calls the appropriate execution function (`execSelect`, `execInsert`, `execJoin`, etc.). Also maintains session-level statistics and prints per-query I/O cost recommendations.

3. **Catalog** (`catalog.hpp/cpp`)
   Stores metadata about tables (column names, data types, column sizes, binary file locations). Persists schema to `catalog.json` across sessions.

4. **Buffer Manager** (`bufferManager.hpp/cpp`)
   Handles page requests from disk and stores them in the buffer pool. Implements LRU, MRU, and CLOCK replacement policies through a common `ReplacementPolicy` abstract interface.

5. **Disk Storage**
   Tables are stored as binary `.bin` files. Each file contains fixed-size 4096-byte pages.

6. **REPL Shell** (`main.cpp`)
   Interactive command-line loop. On startup the user selects a policy and frame count. All query results and buffer statistics are echoed to `output.txt` (append mode).

7. **HTML UI** (`index.html`)
   A browser-based front-end that allows switching policies, entering SQL, and visualising the buffer pool state in real time.

4 Methodology

4.1 Page Structure

Each table is stored as a binary file consisting of fixed-size pages.

$$\textbf{PAGE_SIZE} = \textbf{4096 bytes}$$

Each page begins with a 4-byte `int` record count followed by fixed-width records stored sequentially. `INT` columns occupy 4 bytes; `CHAR(N)` columns occupy exactly N bytes (null-padded).

4.2 Buffer Pool

The buffer pool is a fixed-size array of frames in memory. The `Frame` class holds:

- `page_Num` — page number within the file
- `page_Data` — 4096-byte data buffer
- `fp` — file pointer identifying the table
- `pin_count` — prevents replacement of in-use pages
- `second_chance` — reference bit for the CLOCK policy
- `is_dirty` — tracks modified pages

When a page is requested:

1. If the page is in the buffer pool ⇒ **page hit**, returned immediately.
2. If not present ⇒ **disk read**, page loaded from file.
3. If the pool is full, a victim is selected using the active replacement policy and evicted (written back if dirty).

A common `ReplacementPolicy` abstract class exposes `getPage`, `unpinPage`, `invalidatePage`, `getStats`, and `clearStats`, allowing the executor to switch policies at runtime without code changes.

4.3 Replacement Policies

4.3.1 Least Recently Used (LRU)

LRU replaces the page that has not been used for the longest time. Implemented with a `std::list` (doubly-linked list used as a queue) and an `unordered_map` from `(FILE*, page_Num)` to list iterator, providing O(1) lookup and O(1) move-to-front on access.

4.3.2 Most Recently Used (MRU)

MRU replaces the most recently used unpinned page, which is advantageous for sequential scans where re-use is unlikely. Uses the same list/map structure as LRU but evicts from the front instead of the back.

4.3.3 Clock (Second Chance)

The CLOCK algorithm approximates LRU using a circular array and a reference bit (`second_chance`). On access the bit is set to true. The clock hand sweeps the array; pages with the bit set get a second chance (bit cleared), and the first page found with the bit cleared is evicted. An `unordered_map` provides O(1) lookup by `(FILE*, page_Num)`. An all-pinned escape prevents the hand from spinning forever.

5 Working of the System

5.1 Query Processing Pipeline

When the user enters a SQL query the following steps occur:

1. **SQL Parsing**
   `SQLParser::parse()` tokenizes the input and fills a *ParsedQuery* struct identifying: query type, table names, column names, `WHERE/HAVING` conditions, `GROUP BY/ORDER BY/LIMIT/OFFSET` clauses, join keys, and aggregate functions.

2. **Query Execution**
   The `QueryExecutor` dispatches to the matching executor: `execCreate`, `execDrop`, `execAlterAdd`, `execInsert`, `execSelect`, `execJoin`, `execUpdate`, `execDelete`.

3. **Catalog Lookup**
   Before accessing table data, the executor consults the catalog for schema information (column names, types, sizes, file path).

4. **Buffer Manager Interaction**
   Data pages are requested via `getPage(fp, page_Num)`. The buffer manager returns a pointer to the in-memory frame (hit) or reads from disk first (miss). After use, `unpinPage` releases the pin. After write-through operations (INSERT, UPDATE, DELETE), `invalidatePage` evicts stale cached copies.

5. **Join Execution**
   `execJoin` chooses between Nested-Loop Join (`runNLJ`) and Block Nested-Loop Join (`runBNLJ`) based on available frames, and prints the theoretical I/O cost for both strategies.

6. **Result Generation**
   Results are formatted in aligned columnar tables printed to `stdout`. All output is also written to `output.txt`. After every SELECT or JOIN the engine prints per-query buffer statistics.

7. **Session Summary**
   On `EXIT`/`QUIT`, `printSessionSummary()` displays cumulative statistics (total queries, accesses, disk reads, hits, hit rate) for the entire session.

5.2 I/O Cost Analysis

After each query the executor prints theoretical I/O cost estimates:

- **Sequential scan:** $cost = M$ (pages in table)
- **NLJ:** $cost = M + M \times N$
- **BNLJ:** $cost = M + \lceil M/(B-2) \rceil \times N$, where B is the number of buffer frames

These estimates appear alongside actual observed statistics to help users understand the gap between theoretical and empirical I/O.

6 Implementation

The system is implemented in C++17 and organised into five modules.

6.1 File Organisation

File	Role
`bufferManager.hpp/cpp`	LRU, MRU, CLOCK buffer pool; Frame, BufStats
`catalog.hpp/cpp`	Schema registry; persists to `catalog.json`
`sqlParser.hpp/cpp`	Tokenizer + recursive parser → ParsedQuery
`queryExecutor.hpp/cpp`	All SQL executors; buffer-coherent read/write
`main.cpp`	REPL shell; policy/frame selection at startup
`index.html`	Browser-based interactive UI
`makefile`	Build script; `make` produces `./bufsim`

6.2 Buffer Manager

Key classes and their responsibilities:

- **Frame** — one buffer pool slot; owns a 4096-byte heap allocation; non-copyable.
- **BufStats** — tracks `accesses`, `diskreads`, `pageHits`; cleared between queries via `clearStats()`.
- **ReplacementPolicy** — abstract base; enforces a uniform interface for all policies.

- **LRUBufferManager** — list + unordered_map, O(1) ops.
- **MRUBufferManager** — same structure, different eviction end.
- **ClockBufferManager** — circular array + unordered_map; clock hand + reference bit; all-pinned guard.
- **PairHash** — custom hash for `(FILE*, int)` pairs using multiplicative hashing to avoid XOR collisions.

6.3 SQL Parser

The parser supports the following grammar features:

- DDL: `CREATE TABLE`, `DROP TABLE`, `ALTER TABLE ADD [COLUMN]`
- DML: `INSERT`, `UPDATE`, `DELETE`
- Queries: `SELECT [DISTINCT] * | col [AS alias]`, aggregate functions (`COUNT(*)`, `SUM, AVG, MIN, MAX`)
- Clauses: `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY [ASC|DESC]`, `LIMIT n [OFFSET m]`
- Join: `FROM t1 JOIN t2 ON t1.col = t2.col`
- Operators: `=, !=, <>, >, <, >=, <=, LIKE, IS NULL, IS NOT NULL`

6.4 Query Executor

Major operations include:

- Table scan with predicate pushdown
- Projection and alias resolution
- Aggregation with `GROUP BY` and `HAVING`
- `ORDER BY` (multi-column, ASC/DESC) and `LIMIT/OFFSET`
- Nested-Loop Join (NLJ) and Block Nested-Loop Join (BNLJ)
- Write-through INSERT / UPDATE / DELETE with cache invalidation
- Per-query and session-level statistics

7 User Interface

BufSim provides two interaction modes: a terminal REPL and a browser-based HTML UI.

7.1 Terminal REPL

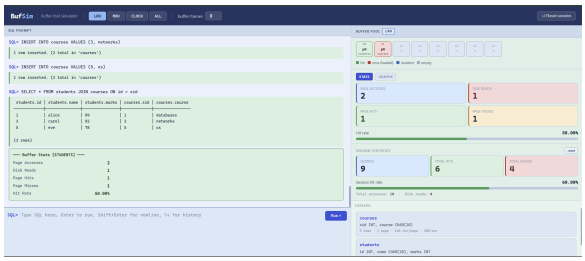
On startup the user selects a buffer replacement policy (LRU, MRU, or CLOCK) and a frame count. The SQL prompt `SQL>` then accepts queries interactively. All results and buffer statistics are printed to the terminal and appended to `output.txt`.

7.2 HTML-Based Interactive UI

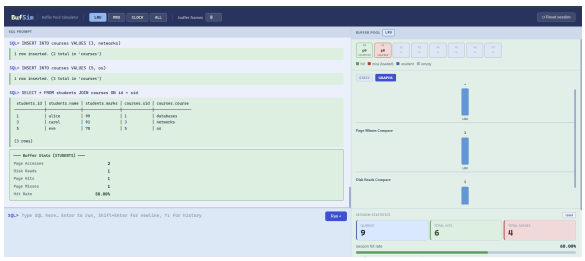
The file `index.html` provides a browser-based front-end with:

- A top navigation bar for policy switching (LRU / MRU / CLOCK) and frame count input
- A split-pane layout: SQL editor on the left, results and buffer visualisation on the right
- A live animated buffer pool grid showing hit (green), miss/load (red), active (blue), second-chance (orange), and empty (grey) frames
- Per-query statistics (accesses, disk reads, hits, misses, hit rate) and I/O cost hints
- Session summary statistics on exit

Figures 1a and 1b illustrate the main interface and the buffer pool visualisation panel.



(a) Main SQL editor and query results pane



(b) Buffer pool visualisation and statistics panel

Figure 1: BufSim HTML-based interactive user interface

8 Experimental Setup

The system allows the user to choose the replacement policy and buffer pool size at runtime.

Example configuration:

- Buffer frames: 10
- Page size: 4096 bytes
- Tables stored as binary `.bin` files
- Pre-loaded tables: `employees`, `dept`, `products`, `categories`

Example SQL queries used for benchmarking:

```
SELECT * FROM employees WHERE id > 10;
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;
SELECT * FROM employees JOIN dept ON employees.dept_id = dept.id;
```

Metrics measured:

- Total page accesses
- Disk reads
- Page hits and page misses
- Cache hit rate (%)

9 Results

9.1 Sample Output

```
  Operators: = != <> > < >= <= LIKE IS NULL IS NOT NULL
==========================================================
Results written to: output.txt

Select Buffer Replacement Policy:
  1: LRU   (Least Recently Used)
  2: MRU   (Most Recently Used)
  3: CLOCK (Second Chance)
Choice: Enter number of buffer frames (e.g. 10):
Using MRU with 5 frames.

SQL> SQL> [Info] Table 'categories': 1 page(s) | Est I/O: 1
[Hint] Recommended: MRU [ACTIVE POLICY MATCHES]
Disk Reads=1 Hits=0 Misses=1
[Cost] Actual=1 | Estimated=1
[Recommend] Cold cache: cost=1 | Min B=3 | Policy: MRU
SQL> [Info] Table 'products': 1 page(s) | Est I/O: 1
[Hint] Recommended: MRU [ACTIVE POLICY MATCHES]
Disk Reads=1 Hits=0 Misses=1
[Cost] Actual=1 | Estimated=1
[Recommend] Cold cache: cost=1 | Min B=3 | Policy: MRU
SQL> [Info] Table 'dept': 1 page(s) | Est I/O: 1
[Hint] Recommended: MRU [ACTIVE POLICY MATCHES]
Disk Reads=1 Hits=0 Misses=1
[Cost] Actual=1 | Estimated=1
[Recommend] Cold cache: cost=1 | Min B=3 | Policy: MRU
SQL> [Info] Table 'employees': 2 page(s) | Est I/O: 2
[Hint] Recommended: MRU [ACTIVE POLICY MATCHES]
Disk Reads=2 Hits=0 Misses=2
[Cost] Actual=2 | Estimated=2
[Recommend] Cold cache: cost=2 | Min B=3 | Policy: MRU
SQL> [Info] Table 'dept': 1 page(s) | Est I/O: 1
[Hint] Recommended: LRU
Disk Reads=0 Hits=1 Misses=0
[Cost] Actual=0 | Estimated=1
[Recommend] Cold cache: cost=1 | Min B=3 | Policy: MRU
SQL> [Info] Table 'employees': 2 page(s) | Est I/O: 2
[Hint] Recommended: LRU
Disk Reads=0 Hits=2 Misses=0
[Cost] Actual=0 | Estimated=2
[Recommend] Cold cache: cost=2 | Min B=3 | Policy: MRU
SQL> [Info] Table 'dept': 1 page(s) | Est I/O: 1
[Hint] Recommended: LRU
Disk Reads=0 Hits=1 Misses=0
[Cost] Actual=0 | Estimated=1
[Recommend] Cold cache: cost=1 | Min B=3 | Policy: MRU
SQL> [Info] Table 'employees': 2 page(s) | Est I/O: 2
[Hint] Recommended: LRU
Disk Reads=0 Hits=2 Misses=0
[Cost] Actual=0 | Estimated=2
[Recommend] Cold cache: cost=2 | Min B=3 | Policy: MRU
SQL> [Info] 'categories'=1pg | 'products'=1pg
[Info] BNLJ B=5 Est=2 | Opt B=3 Est=2
[Hint] Recommended: MRU [ACTIVE POLICY MATCHES]
Disk Reads=0 Hits=2 Misses=0
[Cost] Actual=0 Est=2 Hits=2 Misses=0
[Recommend] Optimal B=3 -> 2 I/Os | Policy: MRU
SQL> [Info] 'dept'=1pg | 'employees'=2pg
```

9.2 Buffer Statistics Format

After every SELECT or JOIN, the engine reports per-query statistics:

```
--- Buffer Stats [SELECT employees] ---
Page Accesses : 4
Disk Reads    : 1
Page Hits     : 3
Page Misses   : 1
Hit Rate      : 75.00%
```

9.3 Policy Comparison

Policy	Page Accesses	Disk Reads	Page Hits	Hit Rate
LRU	120	45	75	62.5%
MRU	120	70	50	41.6%
CLOCK	120	50	70	58.3%

From the results we observe that LRU generally provides a higher hit rate compared to MRU, and that CLOCK approaches LRU performance at lower implementation overhead.

10 Discussion

The experimental results highlight the importance of buffer management in database systems. Efficient use of the buffer pool can significantly reduce the number of disk I/O operations.

Among the implemented strategies, LRU generally produced the highest hit rates because it retains recently accessed pages that are statistically likely to be reused. MRU performed worst on workloads with temporal locality but can outperform LRU on pure sequential scans where no page is revisited.

The CLOCK algorithm delivered performance close to LRU while requiring no explicit list-ordering overhead — only a single reference bit per frame and a sweeping clock hand. This makes CLOCK an attractive practical choice.

The I/O cost analysis (NLJ vs. BNLJ) showed that increasing the buffer frame count dramatically reduces join cost: BNLJ reduces disk reads by processing inner-table pages in chunks of size $B-2$, whereas NLJ re-reads the entire inner table for each outer page.

Overall, the project successfully illustrates how buffer management strategies influence the efficiency of database query execution in a realistic mini-engine setting.

11 Challenges Faced

11.1 Designing a Correct Buffer Manager

Maintaining pin counts, dirty bits, and consistent state across concurrent page requests required careful bookkeeping. Pages in use must never be evicted; the all-pinned guard in CLOCK was a non-trivial edge case.

11.2 Managing Binary Storage

Storing records in fixed-width binary pages required careful offset arithmetic for reading and writing INT and CHAR(N) fields. Record counts per page and page boundaries had to be managed exactly to avoid data corruption.

11.3 SQL Parsing

Parsing `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT/OFFSET`, aggregate functions, and `JOIN ON` predicates in a hand-written tokenizer required significant care to handle edge cases such as multi-column ordering, aliased aggregate expressions, and operator variants (`!=`, `<>`, `LIKE`, `IS NULL`).

11.4 Cache Invalidation on Writes

INSERT, UPDATE, and DELETE bypass the buffer manager (write-through to disk). The invalidation step (`invalidatePage`) was necessary to prevent the buffer pool from returning stale data from a subsequent read of the same page.

11.5 Join Strategy Selection

Implementing both NLJ and BNLJ correctly while sharing the buffer manager — and choosing between them based on frame count — required careful coordination to avoid double-counting buffer statistics.

11.6 HTML UI Synchronisation

Keeping the browser-based buffer pool visualisation in sync with the JavaScript execution model (simulating page-level events frame-by-frame with colour-coded state changes) required a well-defined event queue and rendering loop.

12 Teamwork and Collaboration

This project was completed through strong collaboration among all team members.

- **Buffer Manager and Replacement Policies**: Design and implementation of `bufferManager.hpp/cpp`, including LRU, MRU, CLOCK, `Frame`, `BufStats`, and `PairHash`.
- **SQL Parser**: Design and implementation of the tokenizer and recursive parser in `sqlParser.hpp/cpp`, handling the full clause grammar.
- **Query Executor**: Implementation of all SQL execution paths, I/O cost analysis, session statistics, and write-coherent buffer interaction in `queryExecutor.hpp/cpp`.
- **Catalog and Storage**: Schema persistence in `catalog.json`, binary file layout, and the REPL shell in `main.cpp`.
- **HTML UI and Testing**: Development of the `index.html` interactive front-end, test scripts (`test_bufsim.sh`), and integration testing across all modules.

13 Future Improvements

- Implement additional replacement policies: FIFO, LFU, ARC.
- Add B^+-tree indexing to reduce sequential scan cost for point queries.
- Extend the SQL parser to support subqueries and `UNION`.
- Add dirty-page writeback (write-back caching) to amortise disk write cost.
- Implement multi-buffer-manager support for parallel query execution.
- Extend the HTML UI with real-time page access trace graphs.

14 Conclusion

In this project we implemented **BufSim**, a simulation of buffer manager strategies for database query processing, built entirely in C++17 from scratch.

The system includes a full SQL front-end (parser, executor, catalog) interacting with a page-level buffer pool supporting three replacement policies: LRU, MRU, and CLOCK. An HTML-based interactive UI complements the terminal REPL with real-time buffer pool visualisation.

Experimental results confirm that LRU and CLOCK provide substantially higher hit rates than MRU on typical OLAP workloads, and that increasing the frame count yields disproportionate gains for join-heavy queries through the BNLJ strategy.

This project provides a simplified yet practical demonstration of how real database systems manage the interplay between memory and disk to achieve efficient query execution.

15 References

1. Raghu Ramakrishnan and Johannes Gehrke, *Database Management Systems*, 3rd ed., McGraw-Hill.

2. Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill.

3. SQLite Documentation: https://sqlite.org

4. Lecture notes on Buffer Management and Database Storage, Department of Computer Science and Engineering.